Lab 46: Client tools can't run inside a batch

Topic 4: Prompt Engineering & Structured Output · Task 4.5: Design efficient batch processing strategies

Runs on Simulator 1: Customer Support Resolution Agent · Self-practice

Source: SkillCertPro practice exam Part 1, Q26 (unofficial third-party question)

The question

For a nightly backlog-triage run, this team plans to submit thousands of unresolved conversations through the Message Batches API. The batch inputs include client tool definitions for MCP-backed customer and order systems that are implemented by the team's service. Which design is most appropriate?

A. Move all customer support tools into batch requests as server tools so refunds can complete without follow-up requests.

B. Use batches for initial analysis or tool requests, execute the returned client tool calls, and continue each case in later Messages API requests.

C. Include the client tool schemas in each batch item and let the batch worker execute the backend tools before finalizing responses.

D. Prompt Claude to infer missing customer and order fields whenever a batch item cannot complete the tool loop.

Simulate it live

```
# In the browser: open Simulator 1, then Lab 46
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 46

# In a terminal, from simulators/simulator-1-customer-support/
uv run triage.py --batch
```

What you should see

Rows needing a tool stop at stop_reason: tool_use.

evidence/triage-batch.json records results by custom_id and the note on continuing tool_use rows in follow-up requests.

Open `simulators/simulator-1-customer-support/evidence/` and find it yourself.

The answer

B. Batch the analysis, run returned tool calls, continue in Messages.

Batches can include tool use, and server tools can run inside batch processing. Client tools are different. Claude returns stop_reason: "tool_use" , and your application must execute the tool and send the tool_result in a later request. A single batch request cannot contain that application-mediated loop, because results are retrieved only after processing ends. So use batches for the latency-tolerant first pass. Then match results by custom_id , execute the requested client tools, and submit follow-up requests, synchronous or batched depending on latency needs.

Why the others fail

A. Server tools are Anthropic-hosted. Your own MCP-backed systems cannot become server tools just by being declared that way, and refunds would then run without your application's control.

C. No batch worker executes your client tools.

D. Fabricating customer and order data in a refund workflow.

Check yourself

- results are correlated by custom_id, never by position
- the learner can explain why refunds cannot complete inside the batch

Now change one thing

Copy a file in cases/ to a new CS id, change the customer's message and expected disposition, rerun with --case, then score it with evaluate.py.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.